cqrs.docx

Nexford Learning Solutions

Document Details

Submission ID
trn:oid:::8261:409365669

Submission Date
Aug 25, 2026, 10:18 PM GMT+7

Download Date
Aug 26, 2026, 1:33 AM GMT+7

File Name
cqrs.docx

File Size
24.1 KB

9 Pages

3,379 Words

20,971 Characters

14% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **20** Not Cited or Quoted 12%
  Matches with neither in-text citation nor quotation marks
- **2** Missing Quotations 0%
  Matches that are still very similar to source material
- **3** Missing Citation 2%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

13% Internet sources
6% Publications
11% Submitted works (Student Papers)

Match Groups

- **20** Not Cited or Quoted 12%
  Matches with neither in-text citation nor quotation marks
- **2** Missing Quotations 0%
  Matches that are still very similar to source material
- **3** Missing Citation 2%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

13% Internet sources
6% Publications
11% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
arxiv.org		4%
2	Submitted works	
SETUCW on 2026-08-03		1%
3	Internet	
ojs.aaai.org		1%
4	Publication	
Chaudhuri, Anushree. "Perspectives on Power: Characterizing Public Perceptions ...		<1%
5	Internet	
www.techscience.com		<1%
6	Internet	
aclanthology.org		<1%
7	Internet	
cloud.r-project.org		<1%
8	Submitted works	
Moodle2025 on 2026-08-08		<1%
9	Internet	
lonepatient.top		<1%
10	Submitted works	
St Pauls School London on 2025-04-16		<1%

11 Internet

ouci.dntb.gov.ua <1%

12 Internet

ulb-dok.uibk.ac.at <1%

13 Internet

www.degruyter.com <1%

14 Internet

www.mitpressjournals.org <1%

15 Submitted works

Chattanooga State Community College on 2026-06-19 <1%

16 Submitted works

University of Lancaster on 2026-04-24 <1%

17 Submitted works

University of Essex on 2026-08-25 <1%

18 Submitted works

Curtin University LTI on 2026-04-05 <1%

T-CQRS: Causal State Reconstruction and State-Determination-Aware Retrieval for Distributed RAG in .NET 11

Abstract:
Distributed systems often rely on message logs to keep data consistent. But networks drop, delay, and reorder messages in real-world conditions. When out-of-order events arrive, sorting them on the .NET managed heap triggers repeated garbage collection cycles. This hurts tail latency. Querying these event streams with Retrieval-Augmented Generation (RAG) brings a separate challenge. Embedding search matches text by similarity alone. It cannot tell whether an event is active or superseded. As a result, outdated facts often enter the model prompt. We address these issues with Topological-CQRS (T-CQRS) and State Determination Frontier (SDF) retrieval. T-CQRS re-sequences out-of-order events in memory with value structs and pooled buffers in .NET 11 Preview. In our tests with 10,000 disordered events, no Gen-0 garbage collections were observed. Building on this graph, SDF finds the latest state-changing event and extracts its prerequisite causal ancestors . Across 5,000 benchmark queries with five random seeds, SDF achieved a precision of 1.000 compared to 0.585 for dense vector search (). It maintained complete recall (), removed all observed stale evidence, and reduced prompt tokens by 30.0%. In-memory lookups averaged 18.02 microseconds.
Index Terms:
CQRS, Causal Consistency, Directed Acyclic Graph, State Determination Frontier, Retrieval-Augmented Generation, Garbage Collection Suppression.

Introduction

Many event-driven backends use CQRS to split writes from reads (Young 2010). To keep everything in sync, teams usually pass domain events through a centralized broker like Kafka (Kleppmann 2017). On paper, this maintains a clean sequential log. In real distributed deployments, however, packet drops, routing hiccups, and clock skew scramble event arrival times (Lamport 1978). When services buffer and sort these disordered events on the heap, runtimes like C# .NET trigger repeated garbage collection cycles. Tail latency quickly gets out of hand.

Asking questions over this event log with RAG brings a different headache (Lewis et al. 2020). Say a customer asks: "What is the status of Order 10042?". Vector search only compares text embeddings. It has no idea which events came first or which ones replaced older data. A failed charge (PaymentFailed) and a successful retry (PaymentAuthorized) look practically the same to a vector index. Vector search pulls both into the prompt, leaving the LLM to guess which event reflects reality. Graph-based RAG frameworks connect related text nodes (Erdős et al. 2026), (N. Wang et al. 2025), but they still cannot tell active state updates from superseded ones.

We built two tools to solve this:

Topological-CQRS (T-CQRS): An in-memory causal engine that treats event dependencies as a DAG. Written in .NET 11 Preview using rented ArrayPool<T> buffers and value structs, T-CQRS re-sequences 10,000 out-of-order events with no observed Gen-0 GC collections.

State Determination Frontier (SDF): A state-aware filter. Given a target property , SDF finds the exact event that set the active value, then

pulls its prerequisite causal ancestors .
We test three main questions:
RQ1: Can an in-memory DAG sort disordered events deterministically?
RQ2: Can memory pooling in .NET reduce observed Gen-0 GC collections during high-throughput resolution?
RQ3: Can SDF eliminate stale evidence while preserving the complete prerequisite closure of the authoritative state frontier?
Here is how the rest of the paper is organized. Section 2 covers related work. Section 3 explains T-CQRS. Section 4 defines the State Determination Frontier. Section 5 details our testbed. Section 6 presents experimental findings. Section 7 examines threats to validity, and Section 8 wraps up.
Related Work
Event Sourcing and Distributed CQRS
Event Sourcing and CQRS record domain updates as an append-only event stream (Young 2010). Many systems send these events through brokers like Apache Kafka to keep an exact order (Kleppmann 2017). Over distributed networks, though, packet loss and routing jitter scramble arrival order. When backend services sort these out-of-order events on the heap, memory allocations cause frequent garbage collection cycles that drag down throughput.
Causal Event Ordering and In-Memory Resolution
Instead of enforcing clock order, distributed systems can track causal dependencies between events (Lamport 1978). These dependencies form a Directed Acyclic Graph (DAG) that can be sorted with Kahn's algorithm (Kahn 1962). Most graph libraries allocate linked node objects on the heap. T-CQRS reduces transient array allocations in .NET 11 Preview by using value-type record structs and leasing memory from ArrayPool<T>, sorting events in memory without observed generational GC collections.
GraphRAG and Causal Retrieval
Standard RAG retrieves evidence based on embedding similarity alone (Lewis et al. 2020). This approach works well for static text, but it struggles with mutable event logs. GraphRAG frameworks add entity knowledge graphs to preserve narrative context. CausalRAG (N. Wang et al. 2025) extracts cause-and-effect pairs from text passages. ERAG (Zhang et al. 2026) tracks entity timelines across narrative stories. Still, these systems focus on unstructured prose rather than mutable event-sourced state machines.
Temporal and State-Aware Retrieval
Temporal RAG approaches handle changing information by applying time-decay weights to older passages. Chronofy (Syed et al. 2026) and MemStrata (Yadav 2026) discount evidence exponentially (). ES-GraphRAG (Erdős et al. 2026) focuses on snapshot-consistent streaming graph retrieval, whereas our work explicitly defines a state-determination frontier and its graph-relative prerequisite closure. While time decay helps remove outdated facts, it penalizes foundational setup events like OrderCreated, dropping necessary context needed to answer state queries accurately.
Information-Gain and Entropy-Based Retrieval
Information-theoretic methods trim prompt tokens by removing uninformative passages. Information Gain Pruning (IGP) (Song et al. 2026) ranks evidence by its impact on model uncertainty. BEE-RAG (Y. Wang et al. 2026) uses Shannon entropy (Shannon 1948), (Cover and Thomas 2006) to manage attention over long inputs. We use lexical entropy reduction () as

a secondary compression step once causal boundaries are established.

Research Gap and Taxonomic Comparison

Table [tab:taxonomy] compares T-CQRS and SDF with existing retrieval approaches. Most frameworks focus on semantic similarity or time decay, overlooking causal prerequisites in mutable event streams. SDF addresses this gap by combining allocation-reduced causal ordering with minimal state-determination closures.

T-CQRS Architecture

T-CQRS tracks causal dependencies directly in memory. It structures event history into a Directed Acyclic Graph (DAG). Services can then reconstruct causally ordered state locally, reducing reliance on centralized message ordering during projection.

Causal Node Structure

Most event engines allocate events as heap classes. At high ingestion rates, creating millions of objects triggers frequent garbage collection cycles. We mitigate this by defining events as value-type readonly record struct types in .NET (Microsoft .NET Documentation, n.d.). The event record itself has value-type semantics, while reference-type fields remain separately allocated. Storing records in flat contiguous arrays avoids per-node object header churn and intermediate wrapper allocations.

10 public readonly record struct CausalNode { public required string EventId { get; init; } public required string StreamId { get; init; } public ImmutableArray<string> Prerequisites { get; init; } public DateTime IngestedAt { get; init; } }

Causal DAG: declares and as prerequisites, fixing its logical position regardless of network arrival order.

Mathematical Model of the Causal DAG

We model event history as a directed graph . Vertices represent events, and edges capture causal prerequisites (Lamport 1978). An edge exists when event appears in the Prerequisites array of event , written as .

1 To project state, the engine finds a topological order of using Kahn's algorithm (Kahn 1962). Kahn's topological traversal is with a standard FIFO ready queue; T-CQRS adds deterministic priority ordering among simultaneously ready nodes, whose overhead depends on the ready-set data structure. A DAG can have several valid topological sorts. When multiple nodes become ready at once, T-CQRS resolves ties using a simple rule:

We assume that EventId is unique within the stream. IngestedAt acts only as a tie-breaker for ready nodes; causal order comes strictly from graph edges. The resolver sorts ready nodes by arrival time, breaking remaining ties lexicographically by EventId. This produces a unique, reproducible sequence . In pathological cases with dense dependencies (), resolving deep prerequisite chains takes time. T-CQRS accepts this throughput trade-off under extreme disorder to ensure deterministic causal ordering and suppress Gen-0 collections.

Topological Resolution and Buffer Pooling

Topological sorting often creates temporary lists that cause memory churn. We avoid this by leasing working arrays directly from ArrayPool<CausalNode>.Shared (Microsoft Corporation, n.d.). The topological working set is processed in a leased buffer; the resolved result is materialized separately before the buffer is returned to the pool. No Gen-0 GC collections were observed across all our evaluated ingestion workloads.

public ImmutableArray<CausalNode> Resolve(string streamId)

```
{     CausalNode[] tempArray =
ArrayPool<CausalNode>.Shared.Rent(_nodes.Count);     try {         // In-
place Kahn topological resolution         return
ImmutableArray.Create(tempArray, 0, sortedCount);     }     finally
{         ArrayPool<CausalNode>.Shared.Return(tempArray, clearArray:
true);     } }
```

T-CQRS Event Ingestion and Resolution Pipeline.

State Determination Frontier & Context Selection

T-CQRS produces a causally ordered event sequence. When answering queries with an LLM, however, we only need the events that establish current state values. We formalize this selection through the State Determination Frontier (SDF).

State Model and State Changes

Let be the causal event graph resolved by T-CQRS. An entity state consists of discrete variables .

Definition 1 (State-Changing Transition). An event changes state for variable , written as , if and only if: where is the value of after applying event , and is the projected state before applying along sequence . If event leaves unchanged, .

State Determination Frontier () and Ancestor Closure

Definition 2 (State Determination Frontier). The State Determination Frontier is the latest event in topological sequence that modified :

Definition 3 (Frontier Causal Closure). The causal closure required to reconstruct includes the frontier event and all its prerequisite ancestors:

Theorem 1 (Causal Completeness and Graph-Relative Minimality). Under the assumption that every declared prerequisite of is required for valid state projection along graph , the frontier causal closure is sufficient to reconstruct active state and is minimal with respect to the declared prerequisite graph .

Let be the topological sequence generated by T-CQRS. By Definition 2, is the latest event in satisfying . Because no subsequent event modifies , the active state satisfies . Under the operational assumption that executing requires the replay of its transitive prerequisites, is causally sufficient.

To establish graph-relative minimality on , suppose any node is removed. If , the latest state transition is omitted, leaving an obsolete state projection . If , there exists a prerequisite path in . Removing breaks the required dependency chain in the induced subgraph , violating topological validity in T-CQRS. Thus, no proper subgraph of contains an unbroken causal path to .

Frontier Filtering, Closure Expansion, and Context Selection

The frontier is resolved directly from the authoritative in-memory T-CQRS state projection rather than inferred solely from vector similarity search. Because vector search alone might fail to retrieve early setup prerequisites, our engine expands the candidate set directly from the in-memory graph:

This drops distractor entities and superseded attempts while keeping all needed prerequisites.

State-Determination Context Construction Pipeline.

State Determination Evaluation Metrics

We evaluate context selection and downstream answer correctness using

three primary metrics:

where is the final set of evidence nodes admitted into the prompt context, is the true frontier causal closure, is the state value predicted by the target LLM conditioned on the prompt, and is the ground-truth domain state.

State Determination Precision (SDP) measures context purity. It penalizes distractors and superseded facts. State Determination Recall (SDR) evaluates closure completeness rather than general semantic recall. For SDF, graph expansion guarantees complete closure () whenever is resolved from the authoritative state projection. Our empirical evaluation focuses on frontier accuracy, stale-evidence elimination, reconstructed state accuracy, and prompt token efficiency.

17

Experimental Setup

All experiments ran on a dedicated machine to keep measurements consistent. Table [tab:setup] summarizes the test environment.

Synthetic Stateful Event Stream Benchmark

To evaluate retrieval under realistic distributed faults, we built a synthetic e-commerce event stream generator. It creates 100,000 events across typical order lifecycles, covering checkout, payment attempts, cancellations, and shipments. We injected five specific failure modes:

Out-of-Order Delivery: We shuffled events using Fisher-Yates permutations at a 50% disorder rate.

Stale Supersession: We emitted cancellation events that override previously valid states.

State Contradictions: We introduced conflicting updates targeting the same order property.

Distractor Entities: We added customer profile updates that share vocabulary with order events.

Causal Branching: We added multiple prerequisite parents to test graph convergence.

Evaluation Protocol

We evaluated 1,000 queries per seed across five random seeds (42, 123, 456, 789, and 999), totaling 5,000 test queries. We report mean values and 95% confidence intervals () across all runs. Cohen’s effect sizes are computed from per-query SDP observations using pooled standard deviation across all 5,000 queries relative to dense vector search.

All retrieval baselines were evaluated using the same query set, evidence budget, target LLM, and scoring definitions; baseline-specific retrieval procedures were adapted to expose their final evidence sets to the common SDP, SDR, stale-rate, token, and StateAcc metrics.

T-CQRS vs Traditional CQRS: Latency (ms), Gen-0 GC collections, and heap allocation across ingested nodes.

Experimental Results

Method
SDP
SDR
Stale Rate (%)
Tokens
State Acc. (%)
Latency (ms)
Dense RAG (Top-) (Lewis et al. 2020)
0.585 0.001

0.500 0.000
40.0%
1,200
50.0%
1.20
Dense + Temporal Decay (Syed et al. 2026)
0.400 0.000
0.250 0.000
20.0%
1,200
62.0%
1.40
InfoGain-RAG (IGP) (Song et al. 2026)
0.750 0.000
0.500 0.000
50.0%
960
78.0%
2.10
CF-RAG (ICLR ’26) (Qin et al. 2026)
0.600 0.000
0.625 0.000
60.0%
1,200
86.0%
3.80
ES-GraphRAG (CMC ’26) (Erdős et al. 2026)
0.750 0.000
0.500 0.000
50.0%
960
89.0%
4.50
Proposed T-CQRS + SDF
1.000 0.000
1.000 0.000
0.0%
840
98.5%
2.40
T-CQRS Ingestion Latency and Memory Profiling (RQ1 & RQ2)
We measured event ingestion performance at 1,000 and 10,000 nodes, testing both sequential and out-of-order streams. Table [tab:benchmark] and Fig. 4 report these numbers.
T-CQRS ingested all streams without a single observed Gen-0 GC collection. In comparison, standard list sorting triggered 3 Gen-0 collections at 10,000 nodes. Under severe chaos, internal dictionary growth pushed memory allocations to 5.07 MB. However, leasing buffers through ArrayPool<T> kept transient churn low enough to avoid generational garbage collections entirely. This zero-Gen-0 result comes at a substantial latency cost under extreme disorder: the 10,000-node chaos workload required ms versus ms for the traditional sorter (slowdown due to quadratic prerequisite resolution under complete disorder).

State Determination Retrieval Benchmark (RQ3)
Table [tab:sdf_baselines] and Fig. 5 compare all six retrieval methods across 5,000 queries.
State Determination Precision (SDP) and Recall (SDR) across 6 comparative baselines over 5,000 state queries.
T-CQRS + SDF reached an SDP of . Standard dense RAG achieved only , showing a substantial effect size (). SDF kept full recall () by preserving the complete causal ancestor closure . In contrast, temporal decay dropped recall to because it penalized older setup events like OrderCreated. Furthermore, SDF eliminated all observed stale evidence on our benchmark and reduced prompt tokens by 30.0% (840 versus 1,200 tokens).
In-memory SDF lookups averaged per query. Removing redundant tokens saves roughly in prompt prefill time, using the measured prompt-prefill cost of the evaluated Q4 model configuration (per token). The ratio of saved prefill time to lookup overhead is:
This indicates an estimated ratio between avoided prompt-prefill computation and SDF lookup overhead under the measured workload.
Component Ablation Analysis
We evaluated six architectural variants to assess each layer. Table [tab:ablation] summarizes the results.
Starting from the dense baseline (Variant A), causal tracking alone (Variant B) raised recall to , but still allowed superseded facts into the context. Adding the State Determination Frontier (Variant E) raised recall to and cut stale facts to . The full system (Variant F) achieved perfect precision (), eliminated all observed stale evidence (), and used the fewest tokens ().
Entropy Reduction and Refusal Behavior
Context Entropy Shift per query across Baseline, IG, and IG+Prov modes computed from 30 live experiment traces.
Dense retrieval exhibited a large context entropy shift (). With information gain and provenance tracking (IG+Prov), the shift dropped to (, Wilcoxon test), as shown in Fig. 6. When relevant evidence was absent (IG+Prov_q4.json), the model responded “I don’t know”, adhering strictly to prompt ground truth.
Threats to Validity
Our experimental findings should be interpreted alongside several engineering trade-offs.
Internal Validity
FEVER is built for fact verification over static text rather than stateful event histories. Traditional F1 metrics on FEVER cannot evaluate whether a retrieved fact is active or superseded. For this reason, we used FEVER only for entropy shift diagnostics (). We evaluated state determination accuracy using our 5,000-query benchmark with known causal frontiers (). Our stream generator models e-commerce workflows. Free-form text documents with implicit timestamps may yield different retrieval patterns.
External Validity
All benchmarks ran on a single dedicated machine. T-CQRS removes message broker overhead in a single node, but distributed replication across wide-area networks remains open for future testing. Under complete disorder, our chaos benchmark took ms for events due to cascading prerequisite resolution. This reflects an worst-case cost. Production systems should use indexed graph structures to reach resolution speed.

Finally, our LLM experiments used a 4-bit quantized model (Q4_K_S). Full-precision models may respond differently to prompt noise.

Conclusion and Future Work

This paper introduced Topological-CQRS (T-CQRS) and State Determination Frontier (SDF) retrieval. Together, they address out-of-order event consistency and state blindness in RAG applications.

Our experimental results highlight three key takeaways:

T-CQRS sorts disordered events in memory using value structs and buffer pooling in .NET 11 Preview. No Gen-0 garbage collections were observed across 10,000 ingested nodes, though severe chaos introduced an latency trade-off due to quadratic prerequisite resolution.

SDF isolates the active state transition and reconstructs its causal ancestor closure . Across 5,000 benchmark queries, it achieved perfect precision () and recall (), removing all observed stale facts.

In-memory frontier lookups took while trimming redundant prompt tokens. This yields an estimated ratio between avoided prompt-prefill computation and SDF lookup overhead.

Future work will explore indexed DAG structures () to speed up deep dependency chains and evaluate the framework in multi-node microservice clusters.

Cover, Thomas M, and Joy A Thomas. 2006. Elements of Information Theory. 2nd ed. Hoboken, NJ, USA: Wiley-Interscience.

Erdős, Ferenc, Vijayakumar Varadarajan, Viorel-Costin Banţa, and Stephen Afrifa. 2026. "From Static to Streaming: A Systematic Review and Event-Sourced Framework for GraphRAG in AIOps." Computers, Materials & Continua 88 (3): 4. https://doi.org/10.32604/cmc.2026.081005.

Kahn, Arthur B. 1962. "Topological Sorting of Large Networks." Communications of the ACM 5 (11): 558–62. https://doi.org/10.1145/368996.369025.

Kleppmann, Martin. 2017. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Sebastopol, CA, USA: O'Reilly Media, Inc.

Lamport, Leslie. 1978. "Time, Clocks, and the Ordering of Events in a Distributed System." Communications of the ACM 21 (7): 558–65. https://doi.org/10.1145/359545.359563.

Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, et al. 2020. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." In Advances in Neural Information Processing Systems 33, 9459–74.

Microsoft Corporation. n.d. "ArrayPoolT Class." Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-.buffers.arraypool-1.

Microsoft .NET Documentation. n.d. "C# record types." Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-.us/dotnet/csharp/fundamentals/types/records.

Qin, Huaiyu, Chunyu Wei, Yueguo Chen, and Yunhai Wang. 2026. "Counterfactual Reasoning for Retrieval-Augmented Generation." In Proceedings of the International Conference on Learning Representations (ICLR).

Shannon, Claude Elwood. 1948. "A Mathematical Theory of Communication." Bell System Technical Journal 27 (3): 379–423, July 1948; vol. 27, no. 4, pp. 623–656. https://doi.org/10.1002/j.1538-7305.1948.tb01338.x.

Song, Zhipeng, Yizhi Zhou, Xiangyu Kong, Jiulong Jiao, Xinrui Bao, Xu You, Xueqing Shi, Yuhang Zhou, and Heng Qi. 2026. "Less is More for RAG: Information Gain Pruning for Generator-Aligned Reranking and Evidence

Selection." arXiv Preprint arXiv:2601.17532.

Syed, Muntaser, Marius Silaghi, Sheikh Abujar, and Sharun Akter. 2026. "Chronofy: A Temporal-Logical Decay Architecture for Information Validity in Time-Aware Retrieval-Augmented Generation." arXiv Preprint arXiv:2607.20560.

Wang, Nengbo, Xiaotian Han, Jagdip Singh, Jing Ma, and Vipin Chaudhary. 2025. "CausalRAG: Integrating Causal Graphs into Retrieval-Augmented Generation." In Findings of the Association for Computational Linguistics: ACL 2025, 22680–93. https://doi.org/10.18653/v1/2025.findings-acl.1165.

Wang, Yuhao, Ruiyang Ren, Yucheng Wang, Jing Liu, Xin Zhao, Hua Wu, and Haifeng Wang. 2026. "BEE-RAG: Balanced Entropy Engineering for Retrieval-Augmented Generation." In Proceedings of the AAAI Conference on Artificial Intelligence, 40:33737–45. 40. https://doi.org/10.1609/aaai.v40i40.40664.

Yadav, Neeraj. 2026. "Temporal Validity in Retrieval Memory: Eliminating Stale-Fact Errors for AI Agents over Evolving Knowledge." arXiv Preprint arXiv:2606.26511.

Young, Greg. 2010. "CQRS Documents by Greg Young." [Online]. Available: https://cqrs.wordpress.com/wp-content/uploads/2010/11/cqrs_documents.pdf.

Zhang, Ze Yu, Zitao Li, Yaliang Li, Bolin Ding, and Bryan Kian Hsiang Low. 2026. "Respecting Temporal-Causal Consistency: Entity-Event Knowledge Graph for Retrieval-Augmented Generation." In Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2026), 2017–54. https://doi.org/10.18653/v1/2026.90..90.